Universidade Federal do Rio Grande do Sul
Instituto de Informática
Departamento de Informática Aplicada

Disciplina: INF01086 - Circuitos Digitais
Professor: Mateus Grellert

Laboratório 10: FSM para Multiplicador de Booth

Objetivo:

Implementar a unidade de controle de um multiplicador sequencial baseado no algoritmo de Booth. A parte operativa, ou datapath, será considerada separadamente. O foco desta atividade é traduzir o fluxograma do algoritmo para uma FSM em Verilog, gerando sinais de controle apropriados para comandar registradores, somador/subtrator, deslocamento e contador.

1. Contexto

A multiplicação binária pode ser implementada em hardware de diversas formas. Multiplicadores combinacionais, como array multipliers, calculam o resultado em um único bloco combinacional, mas podem consumir mais área. Multiplicadores sequenciais reutilizam parte do hardware ao longo de vários ciclos de clock, usando registradores, somadores, deslocadores e uma unidade de controle.
O algoritmo de Booth é uma técnica clássica para multiplicação binária com sinal em complemento de dois. Ele reduz sequências de somas repetidas ao observar pares de bits do multiplicador. Em vez de decidir apenas com base no bit atual do multiplicador, o algoritmo analisa o par formado por Q0, o bit menos significativo do registrador do multiplicador, e Q-1, um bit extra que guarda o valor anterior de Q0.

2. Ideia básica do algoritmo de Booth

Considere uma multiplicação M x Q, em que M é o multiplicando e Q é o multiplicador, ambos com N bits em complemento de dois. A implementação sequencial típica usa:

- M: registrador do multiplicando;
- A: acumulador, inicialmente zero;
- Q: registrador do multiplicador;
- Q-1: bit auxiliar, inicialmente zero;
- count: contador de iterações, inicialmente N.

A cada iteração, a unidade de controle observa o par {Q0, Q-1}. A ação escolhida é:

Par {Q0,Q-1}	Operação sobre A	Interpretação
00	Nenhuma operação	Continuação de uma sequência de zeros
01	A <- A + M	Início de uma sequência de uns
10	A <- A - M	Fim de uma sequência de uns
11	Nenhuma operação	Continuação de uma sequência de uns

Após a operação indicada, o conjunto concatenado {A, Q, Q-1} sofre um deslocamento aritmético à direita. O contador é decrementado. Quando count chega a zero, o resultado final está disponível em {A, Q}.

3. Separação entre parte operativa e parte de controle

Neste projeto, o multiplicador deve ser pensado como dois blocos principais:

- Parte operativa (datapath): contém registradores, somador/subtrator, deslocador e contador. Ela executa operações quando recebe sinais de controle.
- Parte de controle (FSM): decide, a cada ciclo, qual operação a parte operativa deve executar. Ela não deve implementar a soma, a subtração ou o deslocamento diretamente.

Importante
Este laboratório é apenas para a parte de controle. Portanto, a FSM deve ser escrita assumindo que existem sinais de status vindos do datapath e sinais de controle enviados para o datapath.

4. Fluxograma do controle

A Figura 1 mostra uma versão de referência do fluxo de controle do algoritmo. A implementação da FSM deve seguir esse comportamento geral, ainda que os nomes dos estados possam variar.

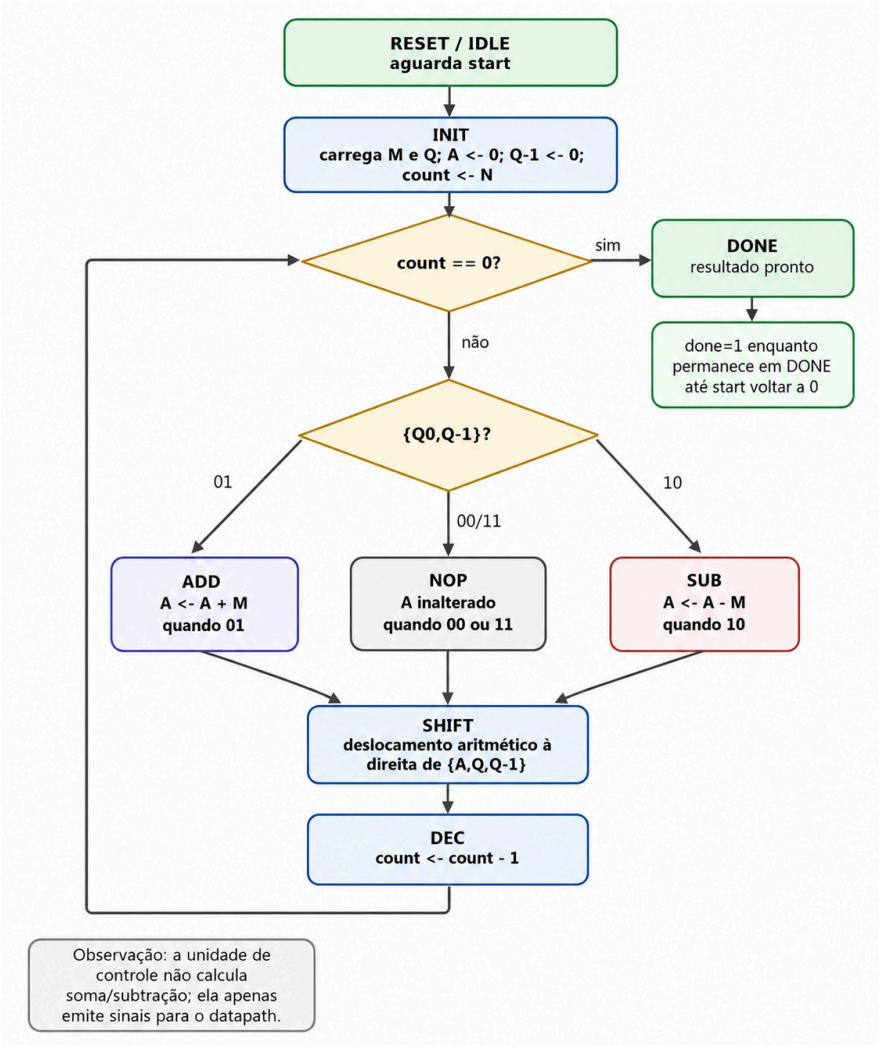


Figura 1 - Algoritmo de Multiplicação de Booth.

5. Descrição textual do algoritmo de controle

1. **IDLE:** Aguardar start = 1. Enquanto start = 0, a FSM permanece parada e done = 0.
2. **INIT:** Inicializar a operação: carregar M e Q, zerar A, zerar Q-1 e carregar count com N.
3. **TEST_COUNT:** Verificar se count == 0. Se sim, ir para DONE. Caso contrário, avaliar {Q0,Q-1}.
4. **TEST_BITS:** Se {Q0,Q-1} = 01, comandar A <- A + M. Se {Q0,Q-1} = 10, comandar A <- A - M. Se for 00 ou 11, não alterar A.
5. **SHIFT:** Comandar o deslocamento aritmético à direita do conjunto {A,Q,Q-1}.
6. **DEC:** Comandar count <- count - 1 e retornar para TEST_COUNT.
7. **DONE:** Indicar done = 1. O resultado está em {A,Q}. A FSM pode permanecer nesse estado até start voltar para 0.

6. Tarefa do laboratório

Implemente, em Verilog, a unidade de controle do multiplicador de Booth como uma máquina de estados finitos. A FSM deve receber sinais de status do datapath e gerar os sinais de controle necessários para comandar as operações do algoritmo.

6.1 Entradas sugeridas para a FSM (todas de 1 bit)

Sinal	Descrição
clk	Clock do sistema.
rst	Reset síncrono ou assíncrono, conforme definido pelo grupo/professor.
start	Solicita o início da multiplicação.
q0	Bit menos significativo do registrador Q.
q_minus_1	Bit auxiliar Q-1.
count_zero	Indica que o contador chegou a zero.

6.2 Saídas sugeridas para a FSM (todas de 1 bit)

Sinal	Descrição
load	Carrega valores iniciais em M, Q, A, Q-1 e count.
op_add	Comanda a operação A <- A + M.
op_sub	Comanda a operação A <- A - M.
op_shift	Comanda o deslocamento aritmético à direita.
op_dec	Comanda o decremento do contador.
done	Indica que a multiplicação terminou.

6.3 Estados sugeridos

Uma implementação simples pode usar os seguintes estados:

- S_IDLE
- S_INIT
- S_CHECK
- S_ADD
- S_SUB
- S_SHIFT
- S_DEC
- S_DONE

Também é aceitável combinar alguns estados, desde que a temporização dos sinais de controle seja coerente com o datapath. Por exemplo, dependendo do datapath, ADD/SUB, SHIFT e DEC podem precisar ocorrer em ciclos separados.

8. Testes mínimos esperados

- Simular a FSM isoladamente usando um testbench simples.
- Aplicar rst e verificar se o estado inicial é S_IDLE.
- Aplicar start = 1 e verificar a sequência INIT -> CHECK.
- Testar as quatro combinações de {Q0, Q-1}: 00, 01, 10 e 11.
- Verificar se op_add é ativado apenas para 01 e op_sub apenas para 10.
- Verificar se a FSM chega em DONE quando count_zero = 1.
- Verificar se done permanece ativo no estado DONE e se a FSM retorna para IDLE quando start = 0.

Instruções de submissão:

1) Crie um diretório LAB**N**, onde **N** representa o número do laboratório
2) Neste diretório, faça upload dos seguintes arquivos
    - Diretório utilizado no projeto Quartus.
    - Arquivo .txt ou .csv descrevendo:
        - Consumo de área da FPGA (Pinos + Logic Elements)
        - Atraso crítico
    - Captura de tela do netlist (Tools > Netlist Viewer > RTL Viewer).
    - Captura de tela da simulação em forma de onda com atraso.
3) Comprima o diretório LAB**N** no formato .ZIP.

Prazo de entrega: até a próxima aula prática.